

Linear Regression

Linear Regression

Hypothesis: $g: X \rightarrow Y \in \{\text{all functions } X \rightarrow Y\}$

Let $X = \mathbb{R}^d, Y = \mathbb{R}$

Parametric models – restriction of class of all functions

- $\exists \mathbf{w} \in \mathbb{R}^d$ – parameters (or weights)
- $g_{\mathbf{w}} = h(\mathbf{x}, \mathbf{w}) \subset \{\text{all functions } \mathbb{R}^d \rightarrow \mathbb{R}\}$

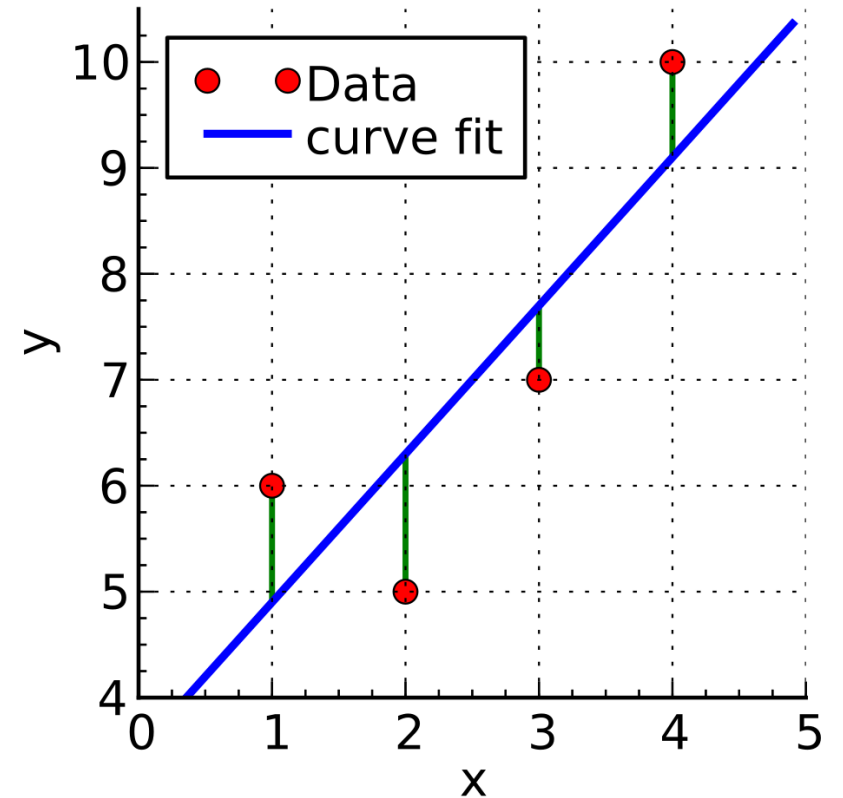
Linear regression:

$$y_i \approx \hat{y} = g_{\mathbf{w}}(\mathbf{x}_i) = \mathbf{x}_i^T \mathbf{w} \text{ for each } i = 1, \dots, n$$

In matrix form:

$$\underbrace{\begin{bmatrix} \hat{y}_1 \\ \vdots \\ \hat{y}_n \end{bmatrix}}_{\hat{\mathbf{y}}} = \underbrace{\begin{bmatrix} \mathbf{x}_1^T \\ \vdots \\ \mathbf{x}_n^T \end{bmatrix}}_{\mathbf{X}} \cdot \underbrace{\begin{bmatrix} w_1 \\ \vdots \\ w_n \end{bmatrix}}_{\mathbf{w}}$$

Where $\mathbf{X} \in \mathbb{R}^{n \times d}$



Linear Regression: Some Agreements

Classical form of linear regression is:

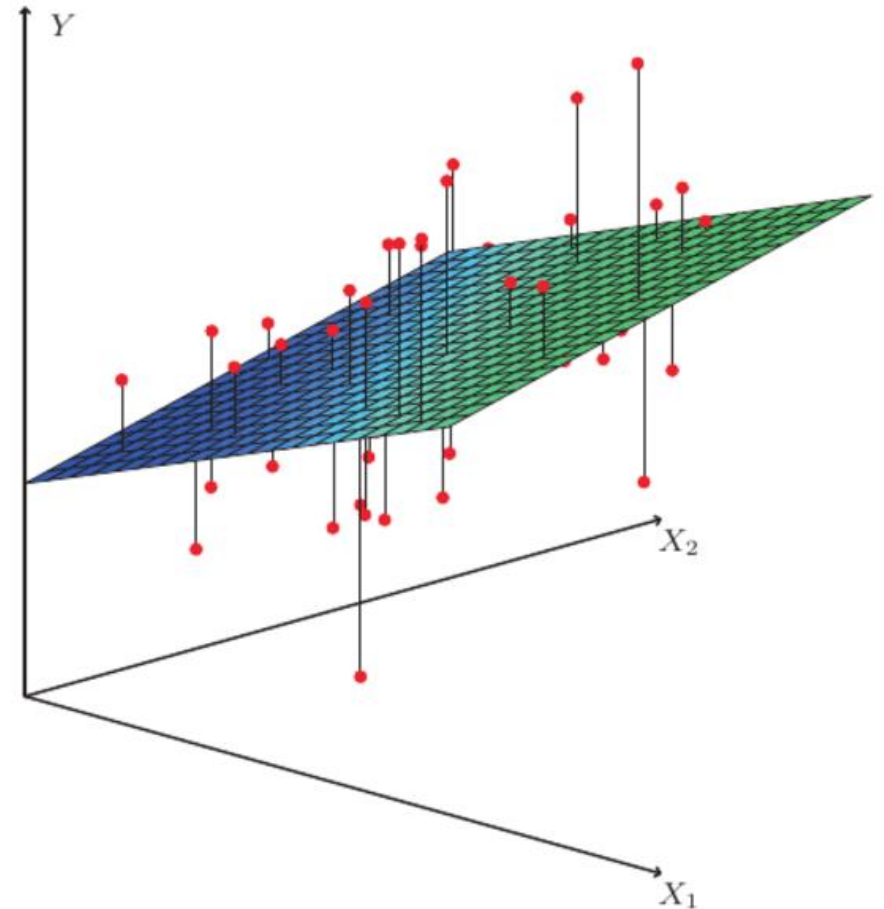
$$\mathbf{y} = \mathbf{X}\mathbf{w} + \mathbf{b}$$

But we "hide" $\mathbf{b}$ in $\mathbf{X}$ and we take this into account in the weight vector.

$$X = \begin{bmatrix} 1 & x_{11} & x_{12} & x_{13} & \dots & x_{1n} \\ 1 & x_{21} & x_{22} & x_{23} & \dots & x_{2n} \\ \vdots & \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_{k1} & x_{k2} & x_{k3} & \dots & x_{kn} \end{bmatrix}$$

- Loss function: L – measure the error between the y and our estimate $\hat{y} = g(\mathbf{x})$
- Risk (true error) of g : $R(h) = \mathbb{E}_{(\mathbf{x}, y) \sim F} [L(g(\mathbf{x}), y)]$, where F is some distribution of our data
- Empirical risk (training error):

$$\hat{R}_{TRAIN}(h) = \frac{1}{n} \sum_{i=1}^n L(g(\mathbf{x}_i), y_i)$$



Linear Regression

Loss function L – how poorly the predictions $\hat{y}$ of the hypothesis match the true output y

$$\mathbf{w}^* = \underset{\mathbf{w}}{\operatorname{argmin}} L(\mathbf{w})$$

Ordinary least squares:

$$L(\mathbf{w}) = \sum_{i=1}^n (x_i^T \mathbf{w} - y_i)^2 = \min_{\mathbf{w}} \|X\mathbf{w} - \mathbf{y}\|_2^2$$

$$\nabla L(\mathbf{w}) = 2X^T(X\mathbf{w} - \mathbf{y}) = 0$$

$$X^T X \mathbf{w} = X^T \mathbf{y}$$

$$\mathbf{w}_{OLS}^* = (X^T X)^{-1} X^T \mathbf{y}$$

Why does this give the minimum?

$$\nabla^2 L(\mathbf{w}) = 2X^T X, \forall \mathbf{w} \Rightarrow \mathbf{w}^T (2X^T X) \mathbf{w} = 2\|X\mathbf{w}\|_2^2 \geq 0$$

Problem

$$\mathbf{X} = \mathbf{U} \mathbf{\Sigma} \mathbf{V}^T$$

We must have that $\mathbf{X}^T \mathbf{X}$ is invertible. Assuming that $\mathbf{X}$ and $\mathbf{X}^T$ are full column rank d , then:

$$\mathbf{X} = \mathbf{U} \begin{bmatrix} \mathbf{\Sigma}_d \\ 0 \end{bmatrix} \mathbf{V}^T$$

where $\mathbf{\Sigma}_d \in \mathbb{R}^{d \times d}$ is diagonal matrix.

$$(\mathbf{X}^T \mathbf{X})^{-1} = \mathbf{V} \mathbf{\Sigma}_d^{-2} \mathbf{V}^T$$

If singular values of $\mathbf{X}$ are close to 0 then $(\mathbf{X}^T \mathbf{X})^{-1}$ have extremely large values.

Problems:

- Numerical stability
- Abnormally high values to the optimal $\mathbf{w}$ solution $\Rightarrow$ bad generalizing to unseen data

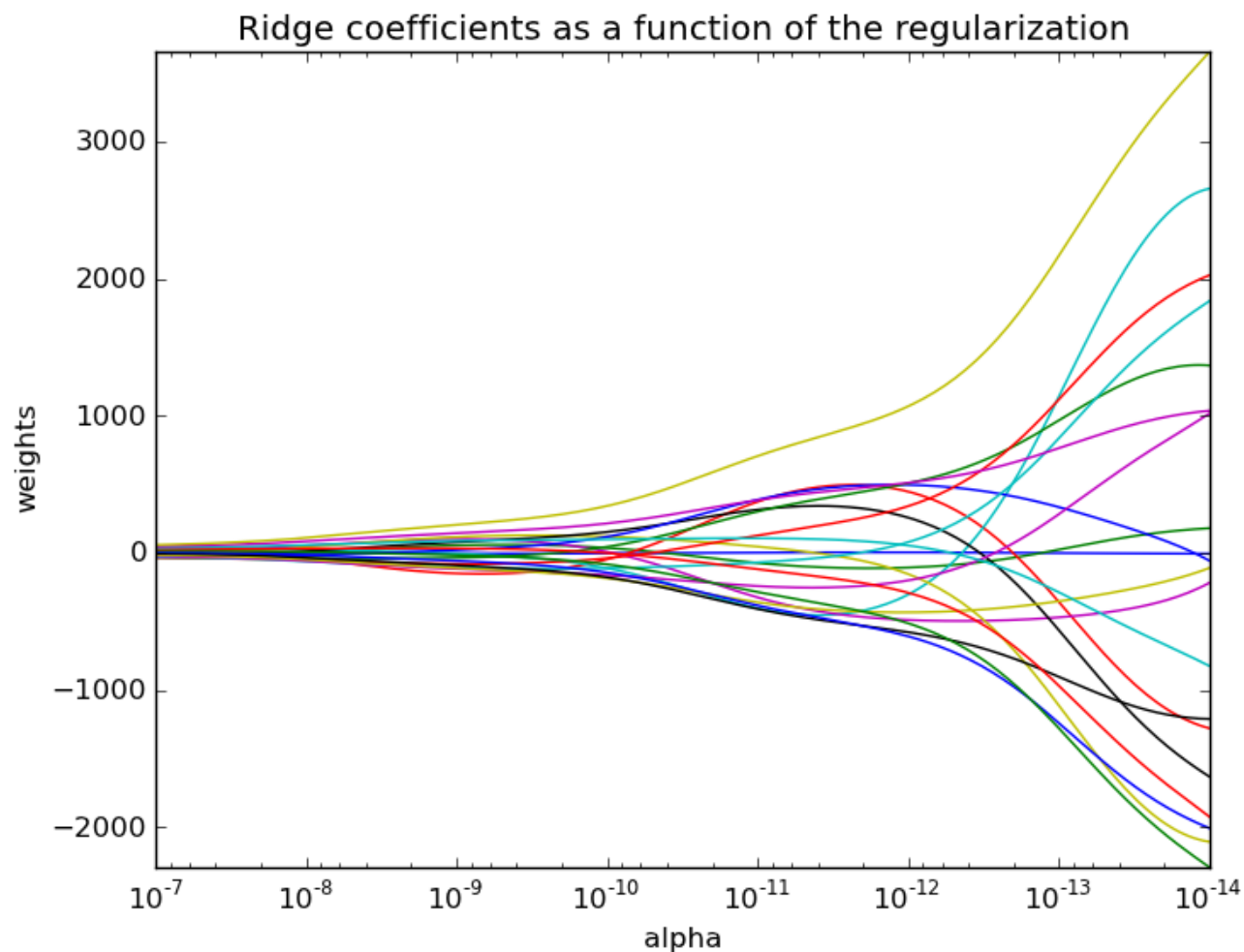
Ridge Regression

Simple solution: penalize the entries of $\mathbf{w}$ from becoming too large

How to do this?

$$\min_{\mathbf{w}} \|\mathbf{X}\mathbf{w} - \mathbf{y}\|_2^2 + \alpha \|\mathbf{w}\|_2^2$$

$$\mathbf{w}_{RIDGE}^* = (\mathbf{X}^T \mathbf{X} + \alpha \mathbf{I})^{-1} \mathbf{X}^T \mathbf{y}$$



Feature Engineering

The true input-output relationship $y = f(\mathbf{x})$ may be nonlinear

Feature map $\phi: \mathbb{R}^d \rightarrow \mathbb{R}^l$

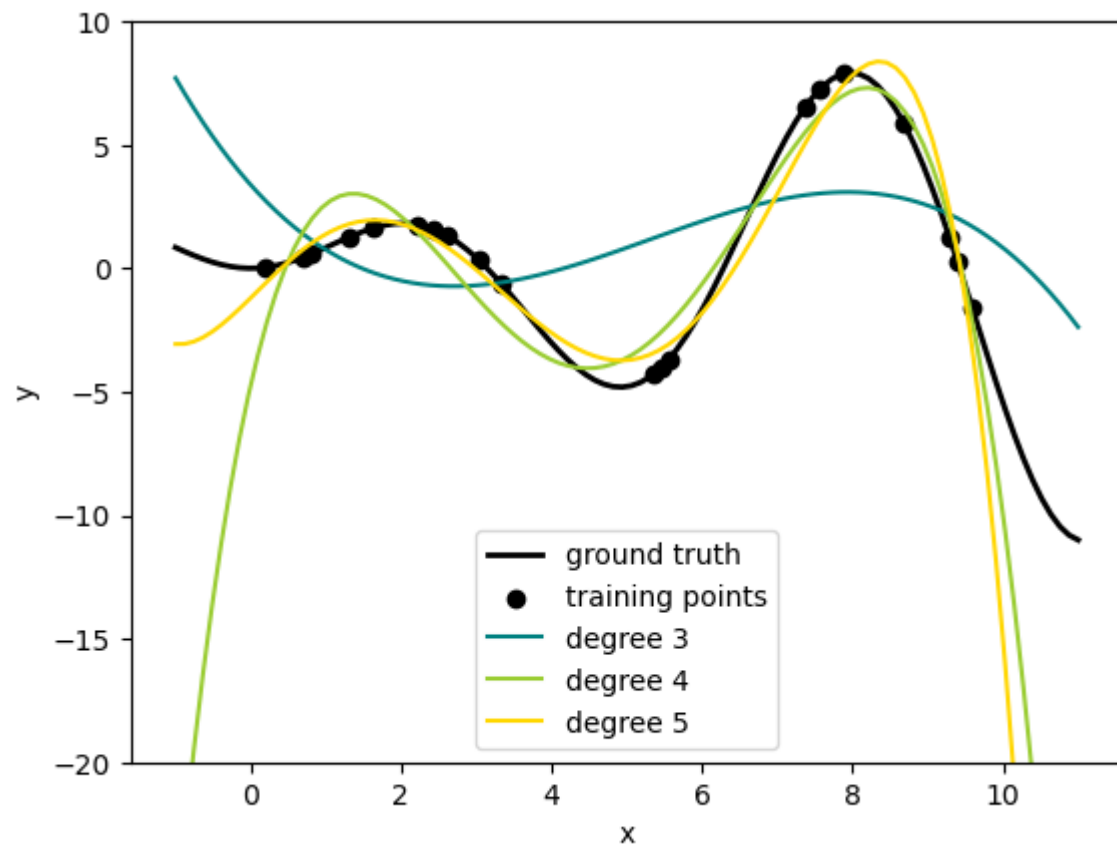
Hypothesis: $g_{\mathbf{w}}(\mathbf{x}) = \phi(\mathbf{x})^T \mathbf{w}$

Example: polynomial

$\mathbf{x} = [x] \rightarrow [1, x, x^2]$

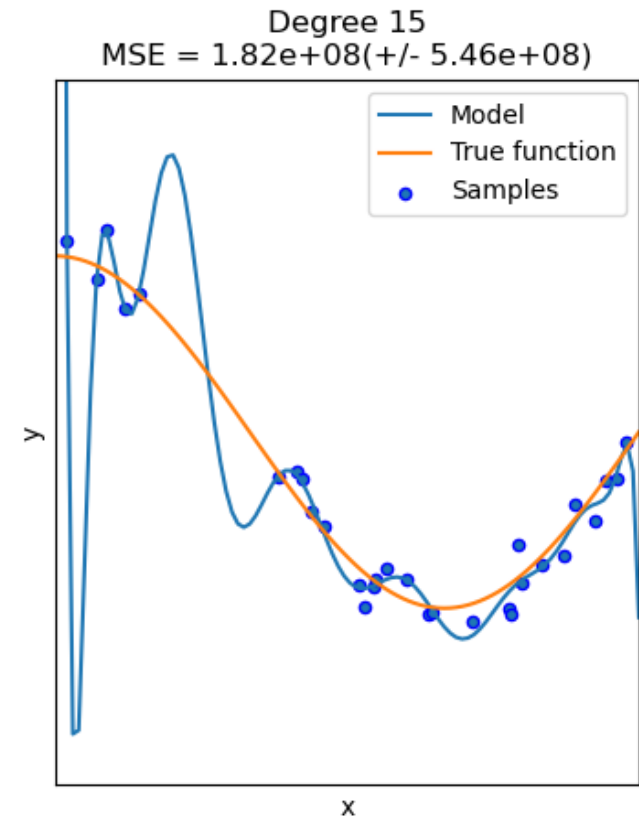
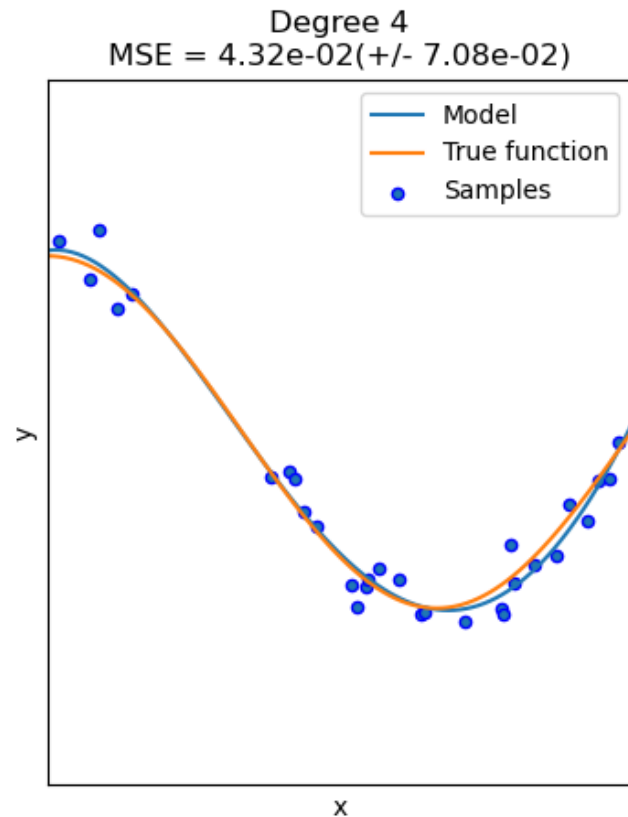
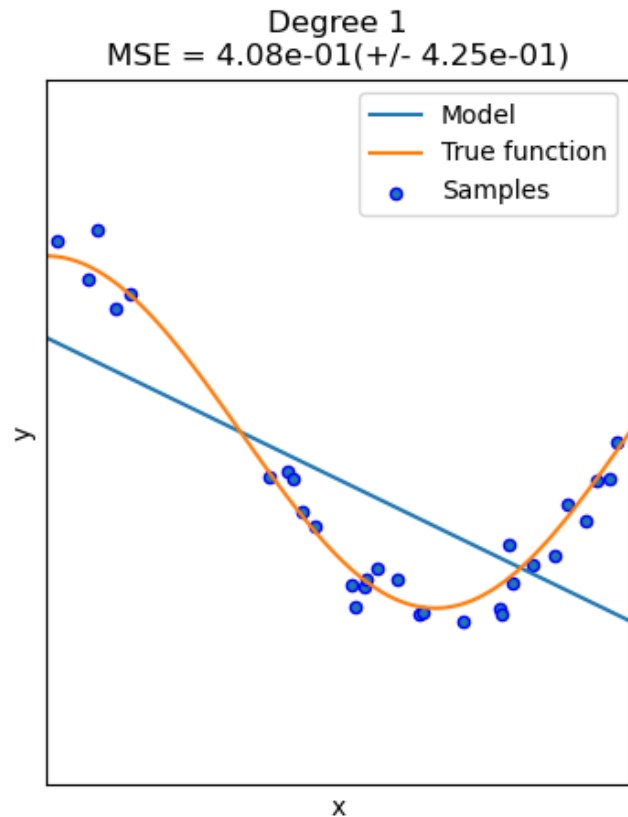
$\mathbf{x} = [x_1, x_2] \rightarrow [1, x_1, x_2, x_1^2, x_2^2, x_1 x_2]$

etc.



Underfitting vs Overfitting

A big reason we care polynomial features is that any smooth function can be approximated arbitrarily closely by some polynomial. For this reason, polynomials are said to be *universal approximators*.



Probabilistic Model

Target – true underlying model:

$$f: \mathbf{x} \mapsto f(\mathbf{x})$$

Dataset:

$$D = \{(\mathbf{x}_i, y_i)\}_{i=1}^n$$

$\mathbf{x}_i$ – is a fixed value, $y_i \in \mathbb{R}$ – is the observation, a noisy version of the true output $f(\mathbf{x}_i)$:

$$Y_i = f(\mathbf{x}_i) + Z_i$$

In most context, we us assume that Z_i is i.i.d Gaussians: $Z_i \sim^{iid} \mathcal{N}(0, \sigma^2)$

$$Y_i \sim^{iid} \mathcal{N}(f(x_i), \sigma^2)$$

Goal: find a hypothesis model g_{θ} that best captures the relationships in the data

Maximum Likelihood Estimation

Goal: find the hypothesis model that maximizes the probability of the data

If we parametrize the set of hypothesis models with $\boldsymbol{\theta}$, then:

$$\hat{\boldsymbol{\theta}}_{MLE} = \underset{\boldsymbol{\theta}}{\operatorname{argmax}} \mathcal{L}(\boldsymbol{\theta}; D) = p(\text{data} = D \mid \text{true model} = g_{\boldsymbol{\theta}})$$

$$\hat{\boldsymbol{\theta}}_{MLE} = \underset{\boldsymbol{\theta}}{\operatorname{argmax}} \mathcal{L}(\boldsymbol{\theta}; \mathbf{X}, \mathbf{y}) = p(y_1, \dots, y_n \mid \mathbf{x}_1, \dots, \mathbf{x}_n, \boldsymbol{\theta})$$

$$\ell(\boldsymbol{\theta}; \mathbf{X}, \mathbf{y}) = \log p(y_1, \dots, y_n \mid \mathbf{x}_1, \dots, \mathbf{x}_n, \boldsymbol{\theta}) = \sum_{i=1}^n \log[p(y_i \mid \mathbf{x}_i, \boldsymbol{\theta})]$$

Each individual term $p(y_i \mid \mathbf{x}_i, \boldsymbol{\theta})$ comes from a Gaussian: $Y_i \mid \boldsymbol{\theta} \sim \mathcal{N}(g_{\boldsymbol{\theta}}(\mathbf{x}_i), \sigma^2)$

$$\hat{\boldsymbol{\theta}}_{MLE} = \underset{\boldsymbol{\theta}}{\operatorname{argmax}} \ell(\boldsymbol{\theta}; \mathbf{X}, \mathbf{y}) = \underset{\boldsymbol{\theta}}{\operatorname{argmax}} - \left(\sum_{i=1}^n \frac{(y_i - g_{\boldsymbol{\theta}}(\mathbf{x}_i))^2}{2\sigma^2} \right) - \text{const}$$

$$\hat{\boldsymbol{\theta}}_{MLE} = \underset{\boldsymbol{\theta}}{\operatorname{argmin}} \sum_{i=1}^n (y_i - g_{\boldsymbol{\theta}}(\mathbf{x}_i))^2, \quad \hat{\boldsymbol{\theta}}_{MLE} = \underset{\boldsymbol{\theta} \in \mathbb{R}^d}{\operatorname{argmin}} \sum_{i=1}^n (y_i - \mathbf{x}_i^T \boldsymbol{\theta})^2$$

Maximum a Posteriori

Goal: find the model, for which the data maximizes the probability of the model

$$\hat{\boldsymbol{\theta}}_{MAP} = \underset{\boldsymbol{\theta}}{\operatorname{argmax}} p(\text{true model} = g_{\boldsymbol{\theta}} \mid \text{data} = D)$$

By Bayes' Rule:

$$\begin{aligned}\hat{\boldsymbol{\theta}}_{MAP} &= \underset{\boldsymbol{\theta}}{\operatorname{argmax}} \frac{p(\text{data} = D \mid \text{true model} = g_{\boldsymbol{\theta}}) \cdot p(\text{true model} = g_{\boldsymbol{\theta}})}{p(\text{data} = D) \leftarrow \text{is a } \mathbf{const}} \\ &= \underset{\boldsymbol{\theta}}{\operatorname{argmin}} -\log p(\text{data} = D \mid \text{true model} = g_{\boldsymbol{\theta}}) - \log p(\text{true model} = g_{\boldsymbol{\theta}})\end{aligned}$$

$p(\text{true model} = g_{\boldsymbol{\theta}})$ is the **prior** over our true model

$$\hat{\boldsymbol{\theta}}_{MAP} = \underset{\boldsymbol{\theta}}{\operatorname{argmin}} - \left(\sum_{i=1}^n \log[p(y_i \mid \mathbf{x}_i, \boldsymbol{\theta})] \right) - \log p(\boldsymbol{\theta})$$

For the prior term $P(\boldsymbol{\Theta})$ we assume that $\theta_j \sim^{iid} \mathcal{N}(\theta_{j_p}, \sigma_p^2)$

$$\hat{\boldsymbol{\theta}}_{MAP} = \underset{\boldsymbol{\theta}}{\operatorname{argmin}} \left(\sum_{i=1}^n (y_i - g_{\boldsymbol{\theta}}(\mathbf{x}_i))^2 \right) + \frac{\sigma^2}{\sigma_p^2} \left(\sum_{j=1}^d (\theta_j - \theta_{j_p})^2 \right)$$

From MAP to Ridge Regression

For the prior term $P(\boldsymbol{\Theta})$ we assume that $\theta_j \sim^{iid} \mathcal{N}(\theta_{j_p}, \sigma_p^2)$

$$\hat{\boldsymbol{\theta}}_{MAP} = \underset{\boldsymbol{\theta}}{\operatorname{argmin}} \left(\sum_{i=1}^n (y_i - g_{\boldsymbol{\theta}}(\mathbf{x}_i))^2 \right) + \frac{\sigma^2}{\sigma_p^2} \left(\sum_{j=1}^d (\theta_j - \theta_{j_p})^2 \right)$$

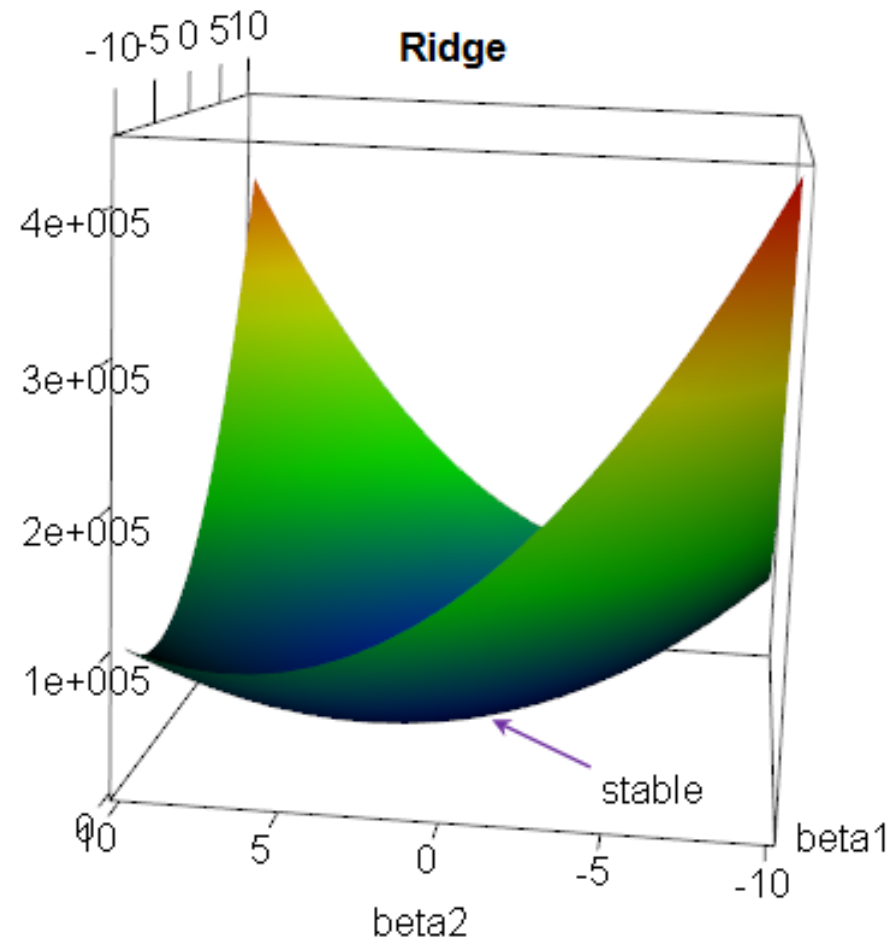
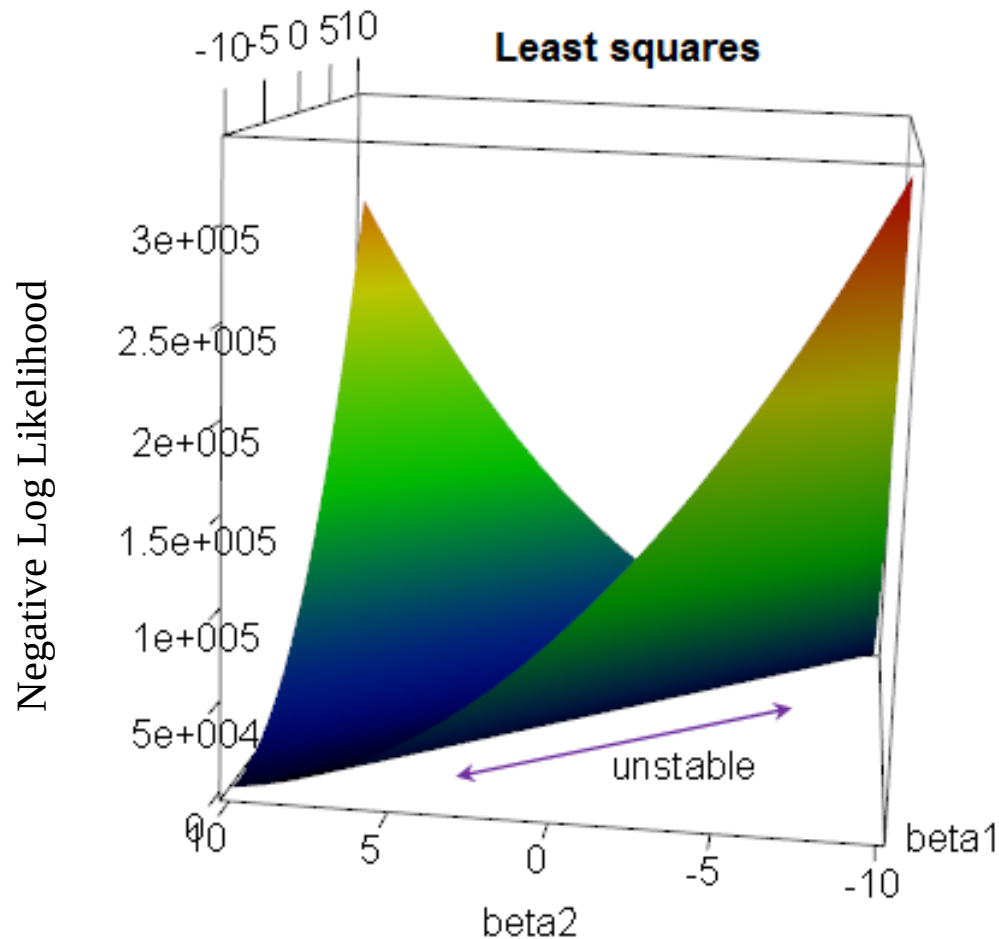
Let $g_{\boldsymbol{\theta}}(x) = \mathbf{x}^T \boldsymbol{\theta}$ and $\theta_{j_p} = 0 \ \forall j$

$$\hat{\boldsymbol{\theta}}_{MAP} = \underset{\boldsymbol{\theta} \in \mathbb{R}^d}{\operatorname{argmin}} \sum_{i=1}^n (y_i - \mathbf{x}_i^T \boldsymbol{\theta})^2 + \frac{\sigma^2}{\sigma_p^2} \sum_{j=1}^d \theta_j^2$$

$$\hat{\boldsymbol{\theta}}_{MAP} = \underset{\boldsymbol{\theta} \in \mathbb{R}^d}{\operatorname{argmin}} \sum_{i=1}^n (y_i - \mathbf{x}_i^T \boldsymbol{\theta})^2 + \alpha \sum_{j=1}^d \theta_j^2, \alpha = \frac{\sigma^2}{\sigma_p^2}$$

Why “Ridge”?

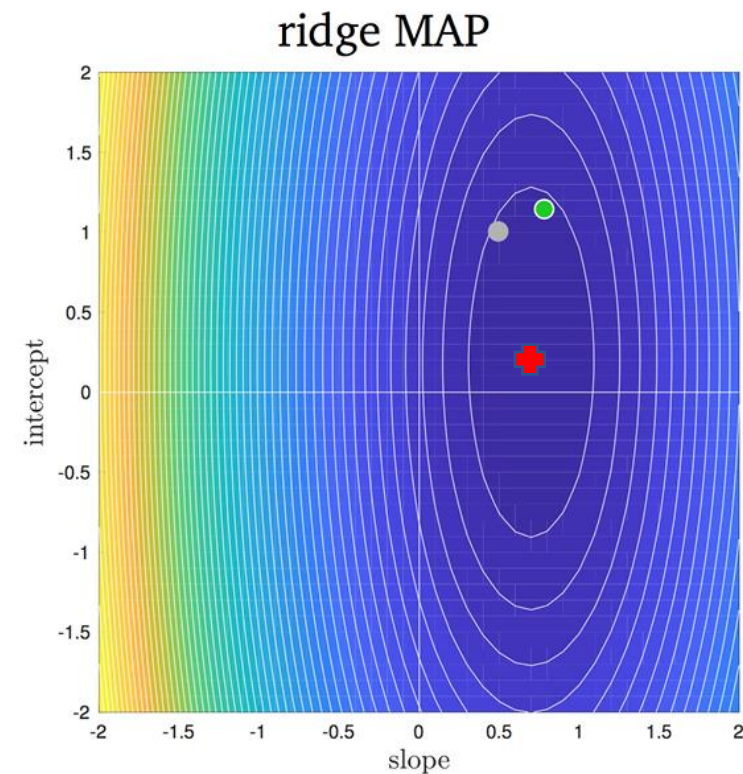
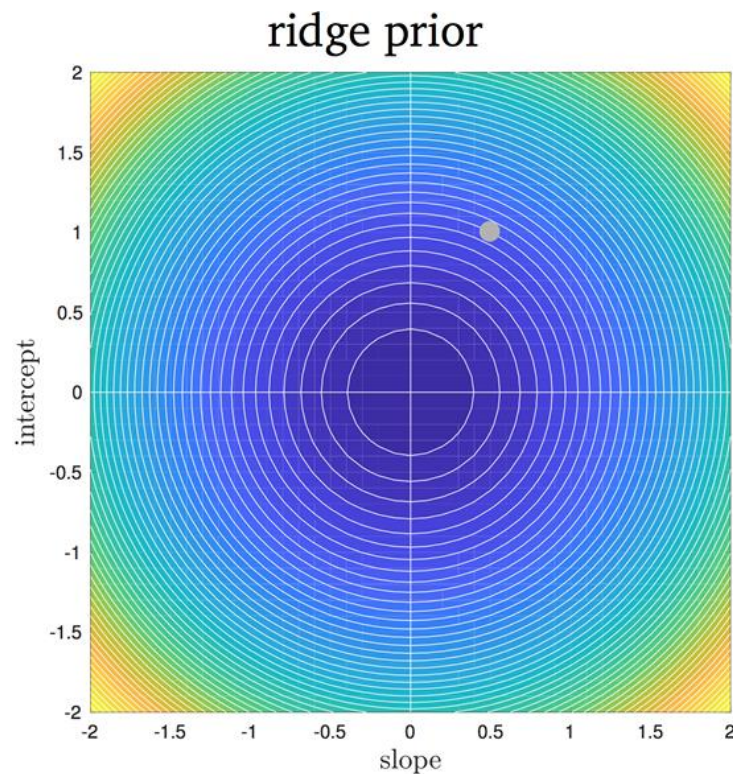
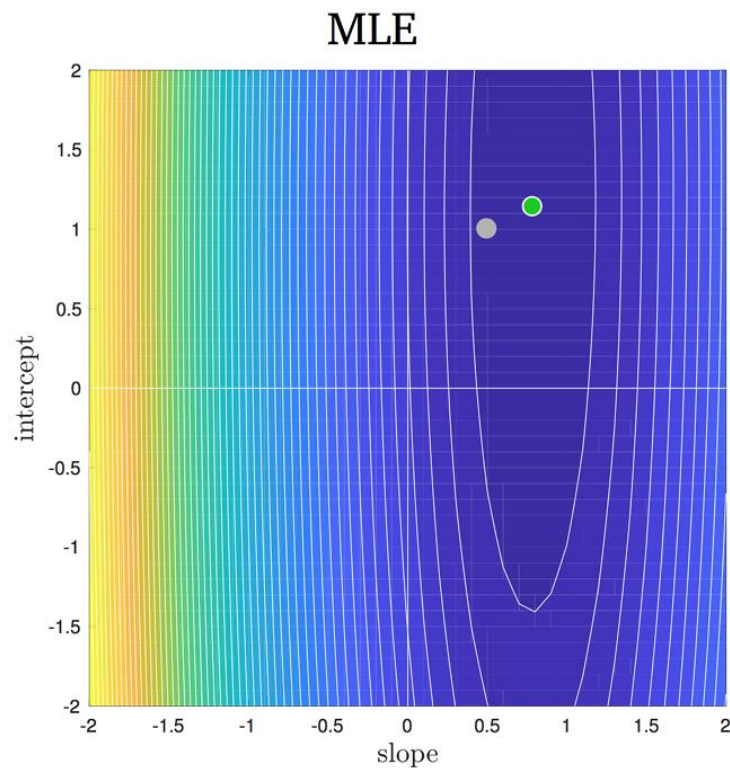
When some features are linearly dependent (can't invert $X^T X$), we have ∞ many equally “good” solutions that form a ridge.



MLE vs MAP

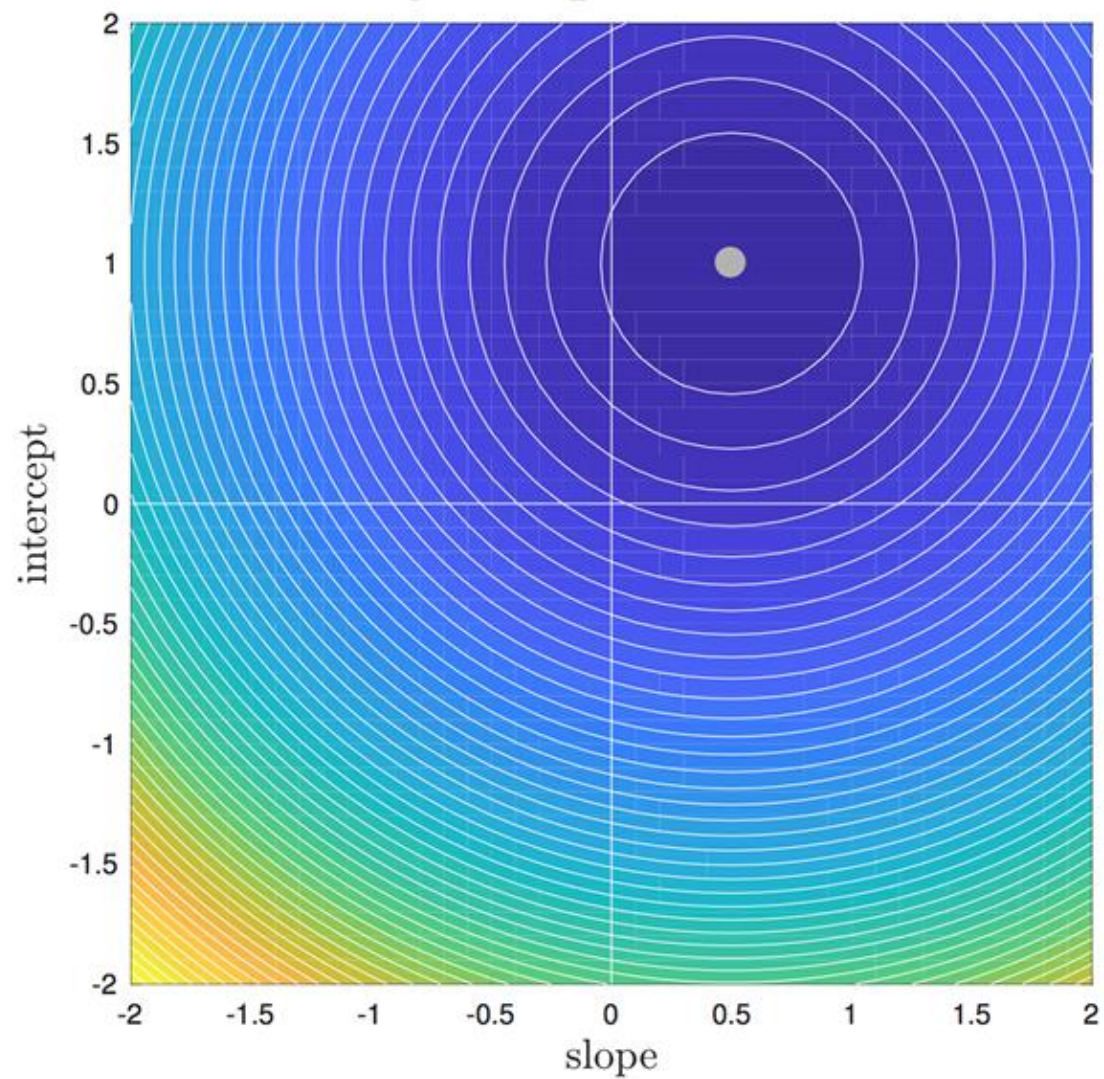
Let $f(x) = \text{slope} \cdot x + \text{intercept}$, where $(\text{slope}, \text{intercept}) = (0.5, 1.0)$

What parameters will MLE and MAP choose when given some dataset D ?

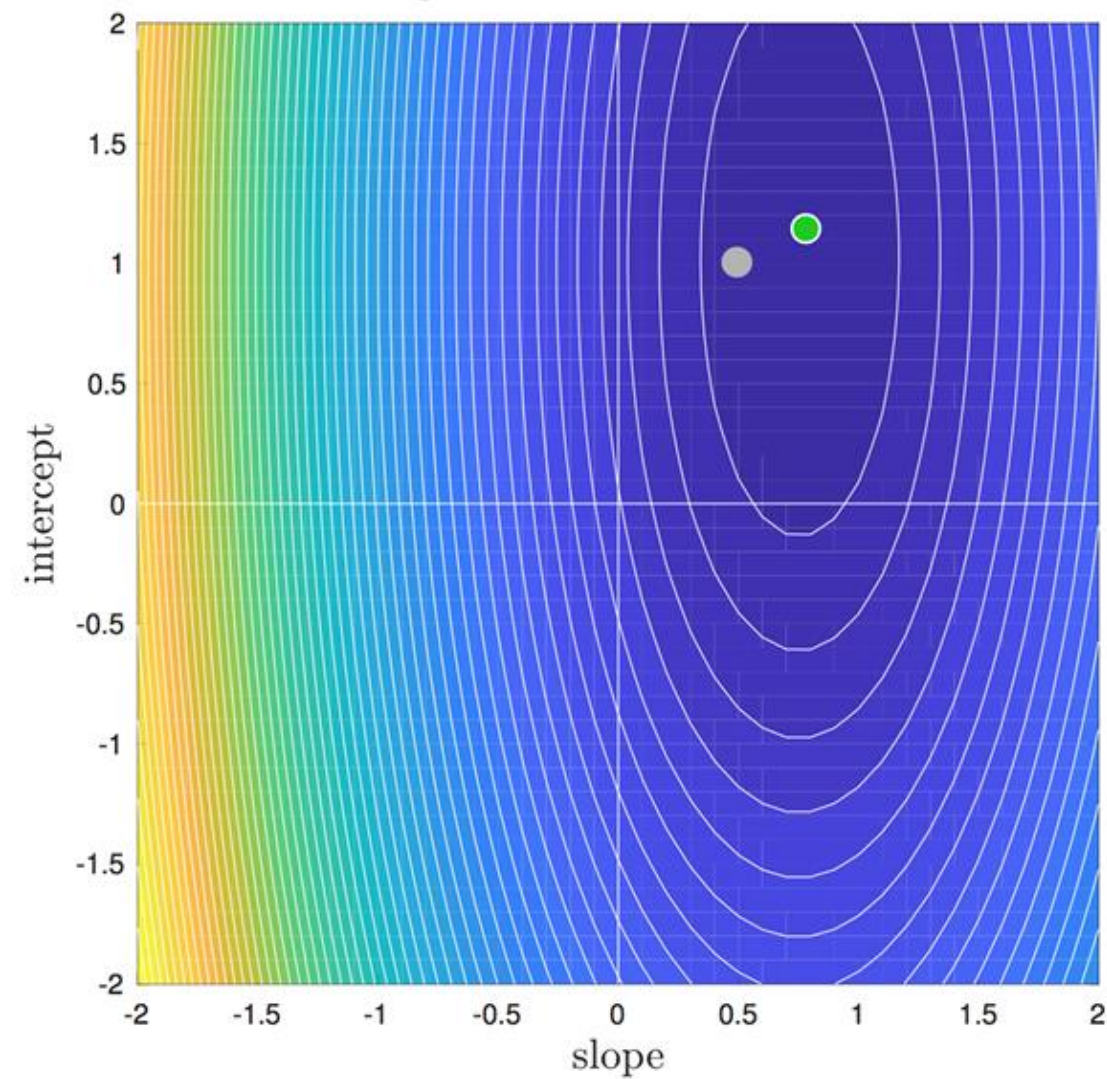


MLE vs MAP

good prior

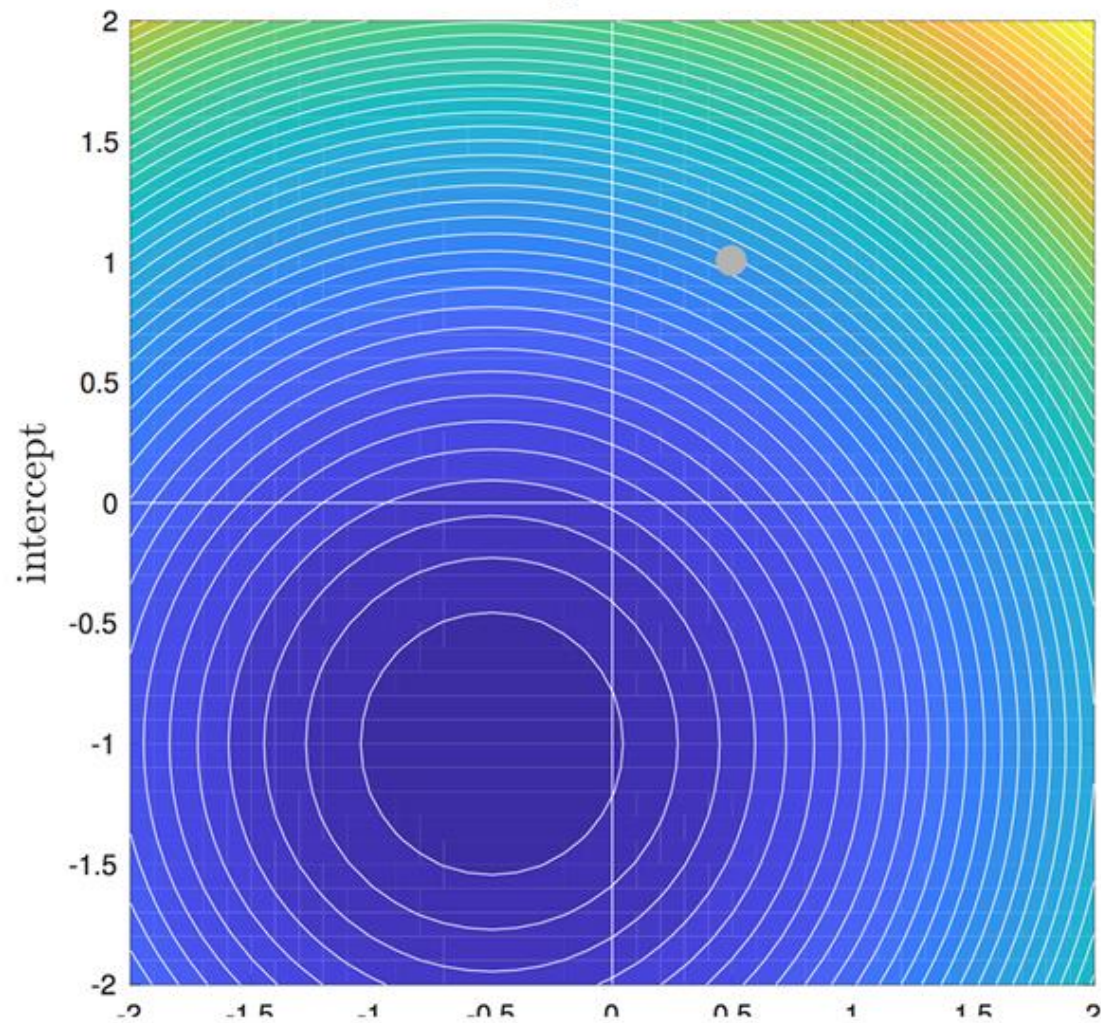


good MAP

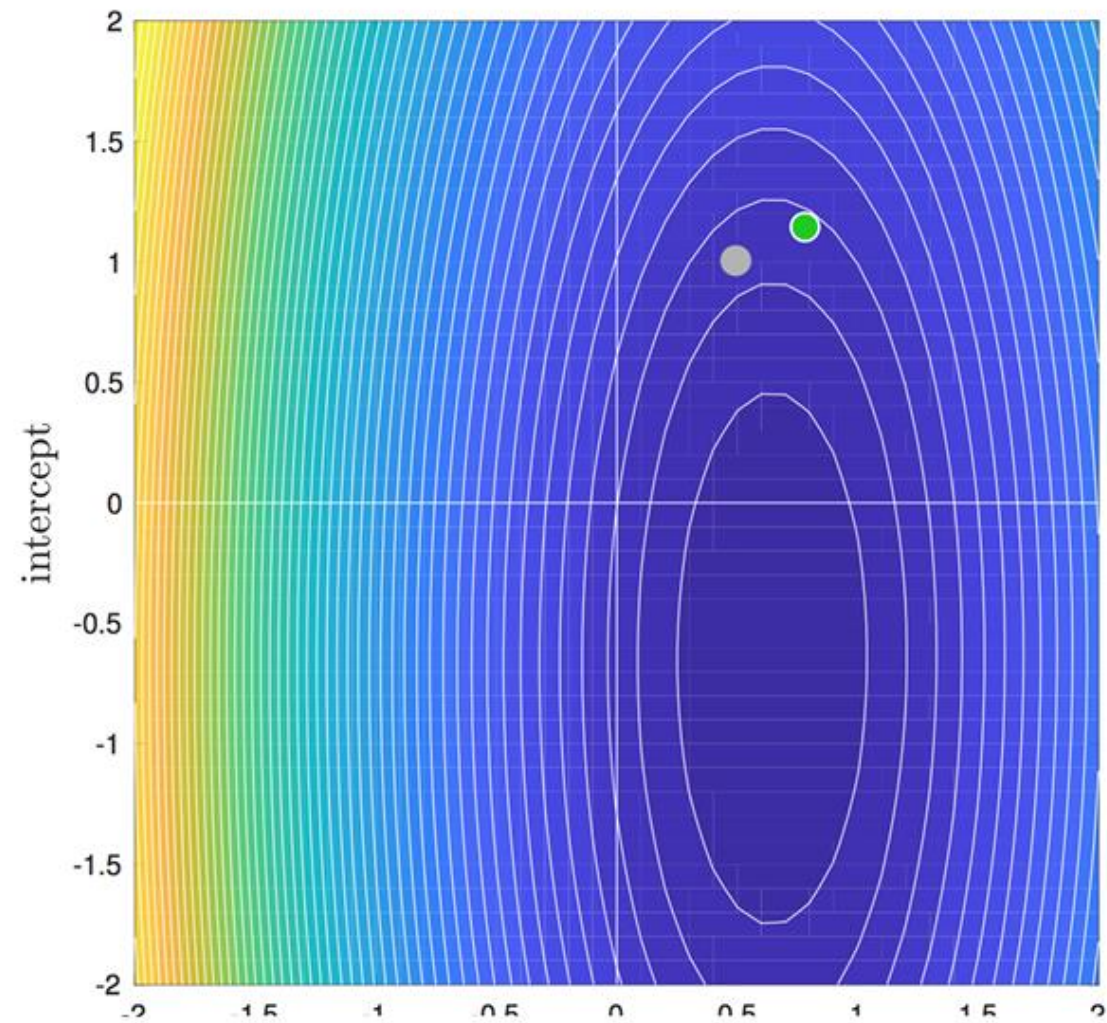


MLE vs MAP

bad prior



bad MAP



Bias-Variance Tradeoff

Target – true underlying model: $f: \mathbf{x} \mapsto f(\mathbf{x})$

Dataset: $D = \{(\mathbf{x}_i, y_i)\}_{i=1}^n$

$\mathbf{x}_i$ – values sampled from random variables $\mathbf{X}_i$, $y_i \in \mathbb{R}$ – is the observation, a noisy version of the true output $f(\mathbf{x}_i)$: $Y_i = f(\mathbf{x}_i) + Z_i$

Hypothesis: $g: X \rightarrow Y$

For some arbitrary test input $\mathbf{x}$, $g(\mathbf{x}; D)$ depends on the random variable D that was used to train g .

$\mathbf{x}$ and $\mathbf{D}$ are completely independent from one another!

How exactly can we measure the effectiveness of a hypothesis model?

Bias-Variance Tradeoff

Goal: for a fixed test point $\mathbf{x}$, evaluate how closely the hypothesis can estimate the noisy observation Y corresponding to $\mathbf{x}$

Our metric:

$$\begin{aligned}\epsilon(\mathbf{x}; g) &= \mathbb{E}[(g(\mathbf{x}; D) - Y)^2], & Y &= f(\mathbf{x}) + Z \\ \mathbb{E}_{D,Y}[(g(\mathbf{x}; D) - Y)^2] &= \mathbb{E}_D[\mathbb{E}_Y[(g(\mathbf{x}; D) - Y)^2 \mid D]]\end{aligned}$$

This metric is difficult to interpret and work with. Decompose it.

$$\begin{aligned}\mathbb{E}(Y) &= f(\mathbf{x}), & \text{Var}(Y) &= \text{Var}(Z), & \mathbb{E}(X^2) &= \text{Var}(X) + \mathbb{E}(X)^2 \\ \epsilon(\mathbf{x}; g) &= \mathbb{E}[(g(\mathbf{x}; D) - Y)^2] = \mathbb{E}[g(\mathbf{x}; D)^2] + \mathbb{E}[Y^2] - 2\mathbb{E}[g(\mathbf{x}; D) \cdot Y] \\ &= (\text{Var}(g(\mathbf{x}; D)) + \mathbb{E}[g(\mathbf{x}; D)]^2) + (\text{Var}(Y) + \mathbb{E}[Y]^2) - 2\mathbb{E}[g(\mathbf{x}; D)] \cdot \mathbb{E}[Y]\end{aligned}$$

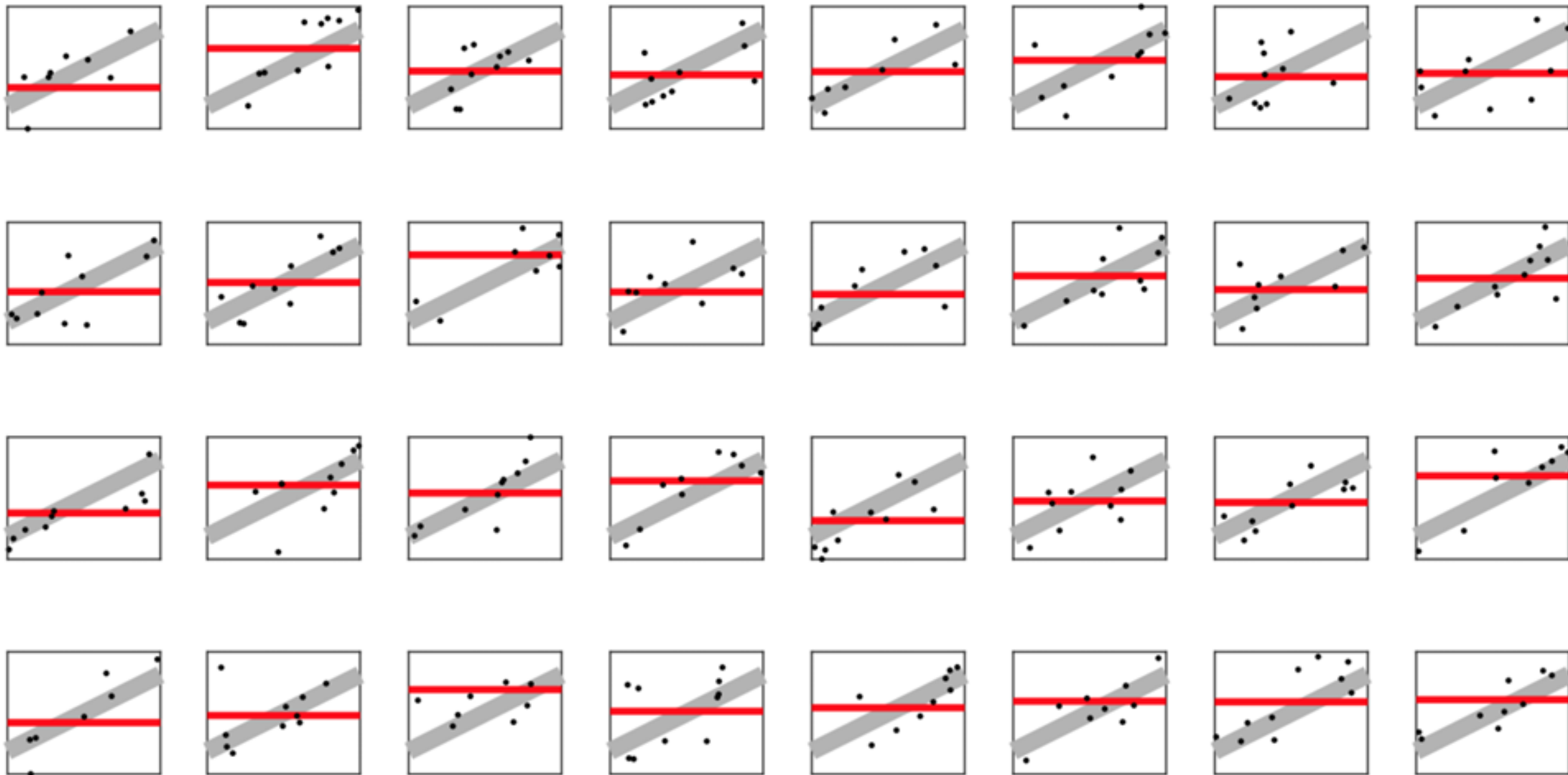
(because for any two independent random variables D and Y , $f_1(D)$ and $f_2(Y)$ are also independent, for any functions f_1, f_2)

Bias-Variance Tradeoff

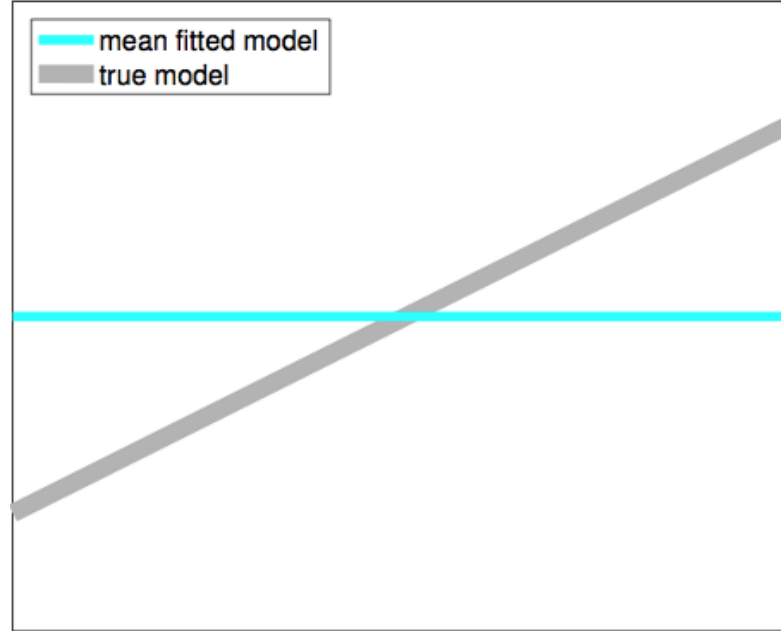
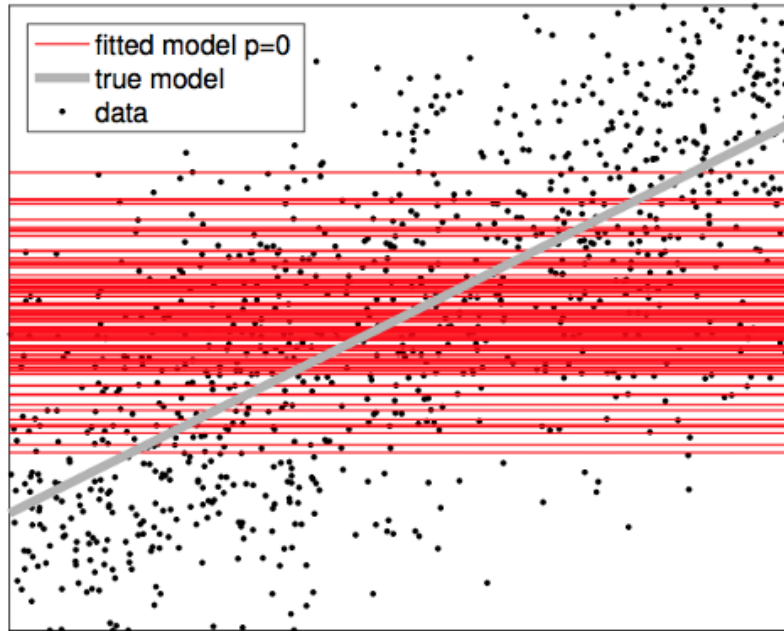
$$\begin{aligned} & (Var(g(\mathbf{x}; D)) + \mathbb{E}[g(\mathbf{x}; D)]^2) + (Var(Y) + \mathbb{E}[Y]^2) - 2\mathbb{E}[g(\mathbf{x}; D)] \cdot \mathbb{E}[Y] \\ &= (\mathbb{E}[g(\mathbf{x}; D)]^2 - 2\mathbb{E}[g(\mathbf{x}; D)] \cdot \mathbb{E}[Y] + \mathbb{E}[Y]^2) + Var(g(\mathbf{x}; D)) + Var(Y) \\ &= (\mathbb{E}[g(\mathbf{x}; D)] - \mathbb{E}[Y])^2 + Var(g(\mathbf{x}; D)) + Var(Y) \\ &= \underbrace{((\mathbb{E}[g(\mathbf{x}; D)] - f(x))^2)}_{\text{bias}^2 \text{ of method}} + \underbrace{Var(g(\mathbf{x}; D))}_{\text{variance of method}} + \underbrace{Var(Z)}_{\text{irreducible error}} \end{aligned}$$

- *Bias² of method*: measures how well the average hypothesis (over all possible training sets) can come close to the true underlying value $f(x)$, for a fixed value of x .
- *Variance of method*: measures the variance of the hypothesis (over all possible training sets), for a fixed value of x . An un-biased method (bias = 0) could have a large variance.
- *Irreducible error*: this is the error in our model that we cannot control or eliminate, because it is due to errors inherent in our noisy observation Y .

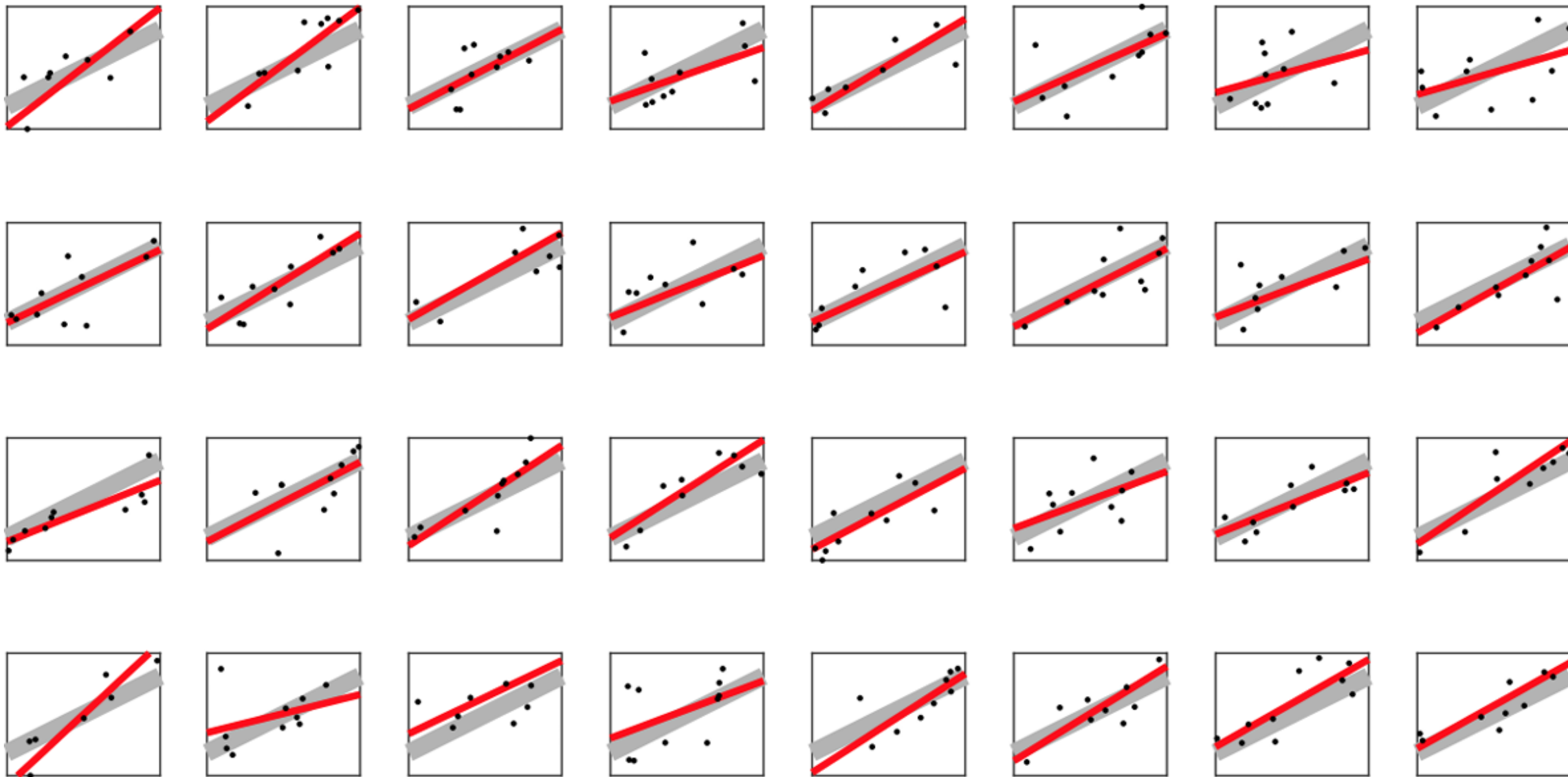
Experiments: $\text{deg} = 0$



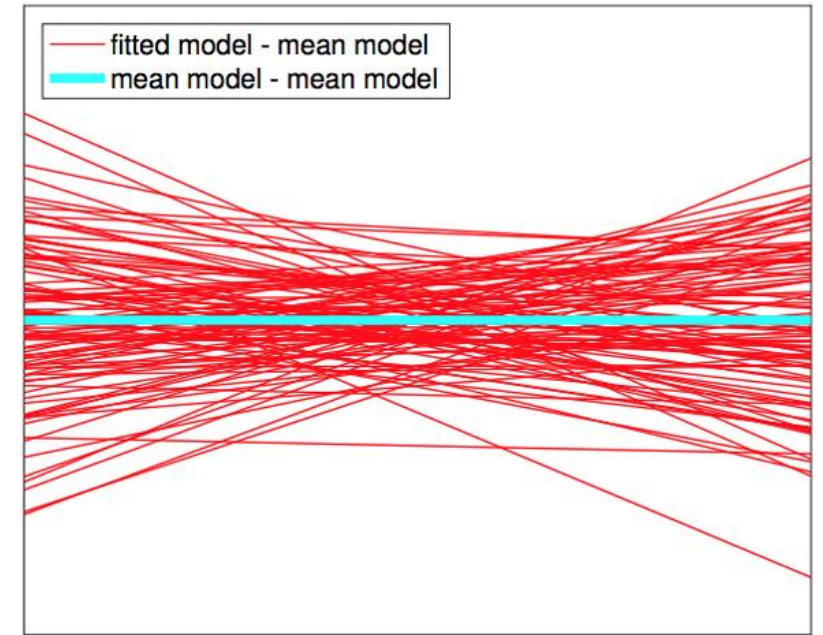
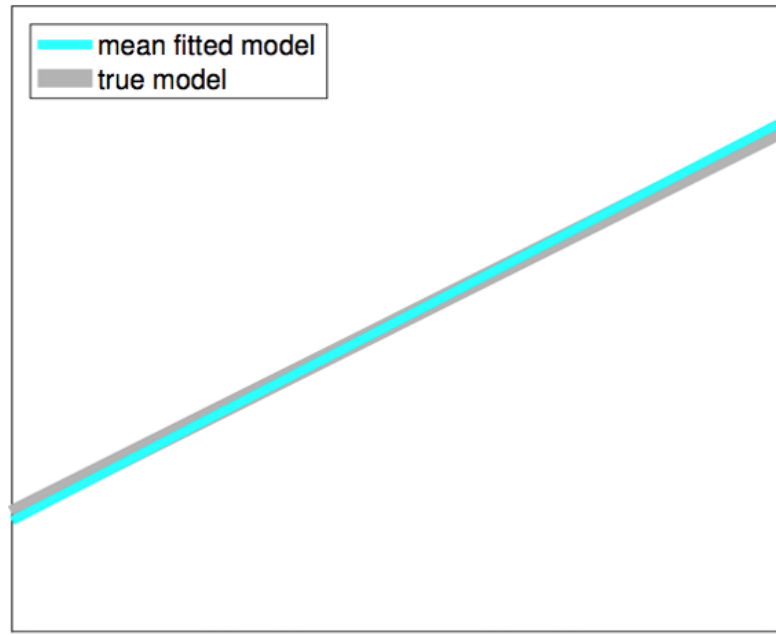
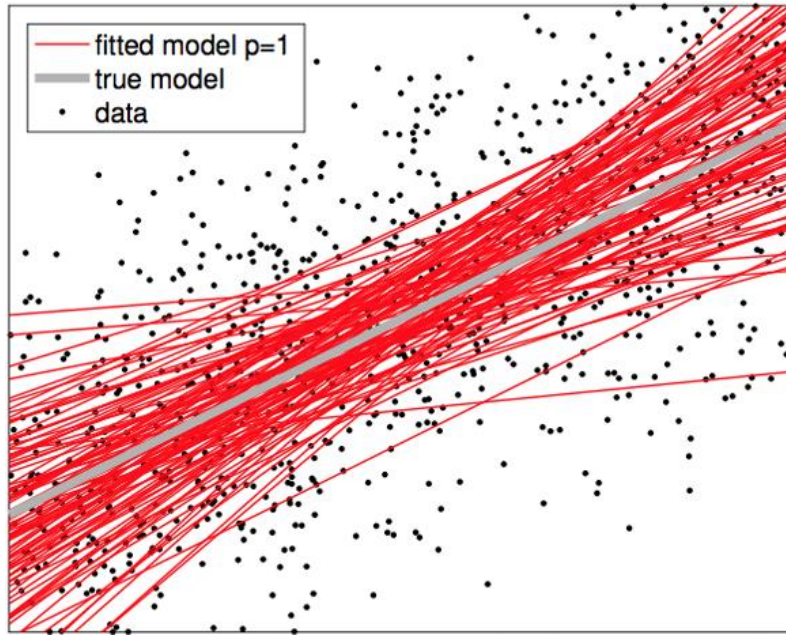
Bias and Variance: $\text{deg} = 0$



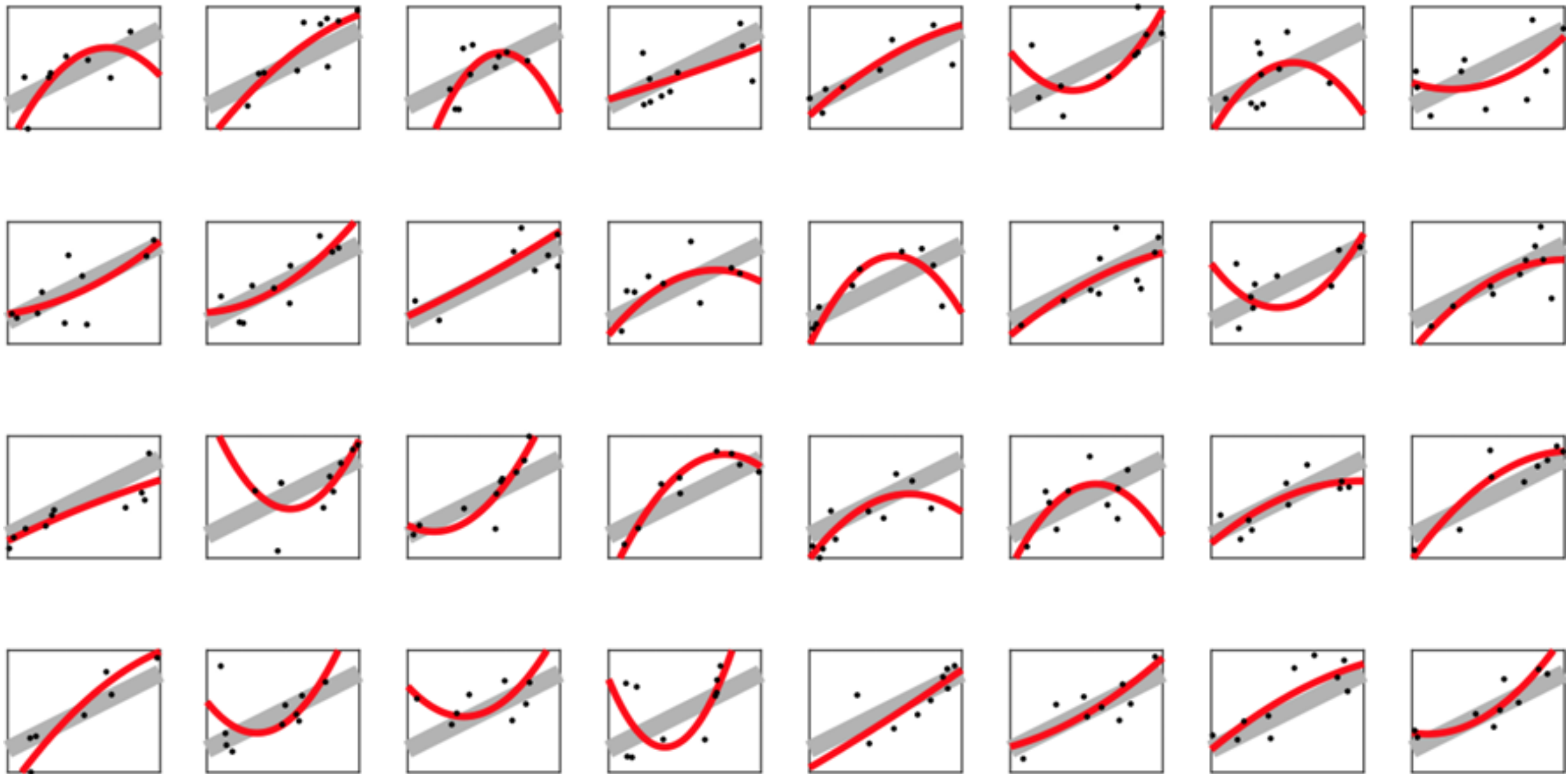
Experiments: $\text{deg} = 1$



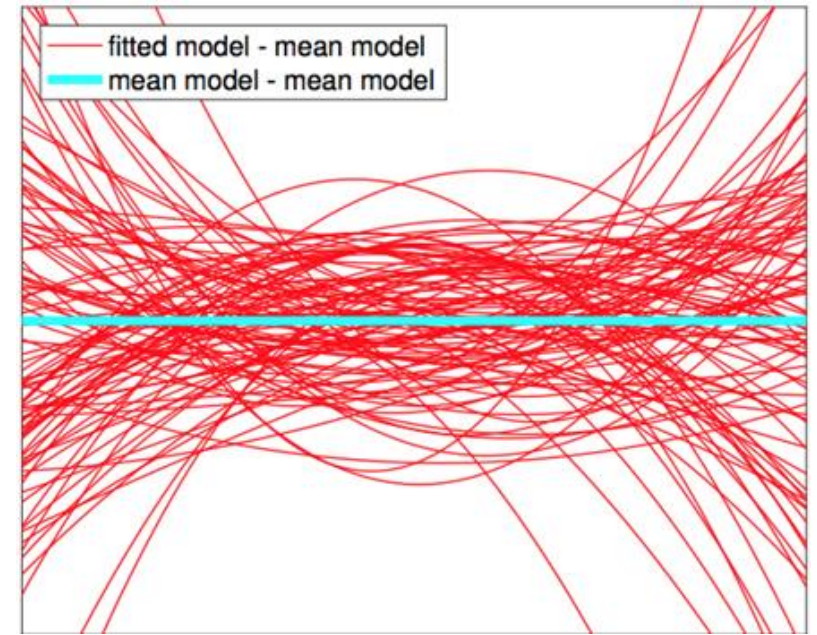
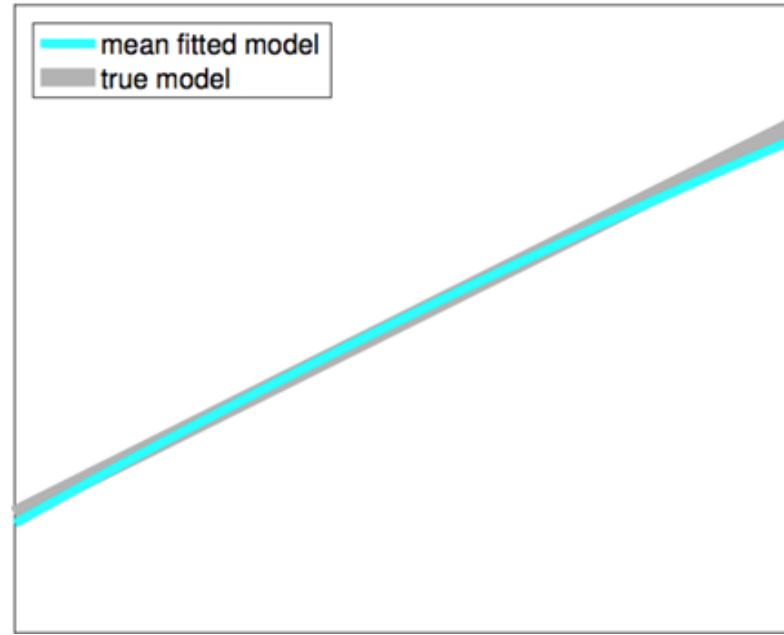
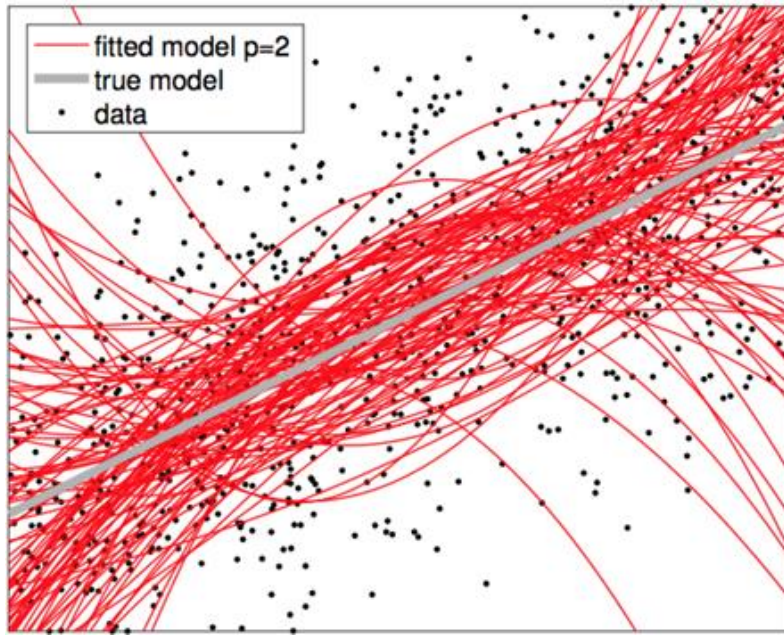
Bias and Variance: $\text{deg} = 1$



Experiments: $\text{deg} = 2$

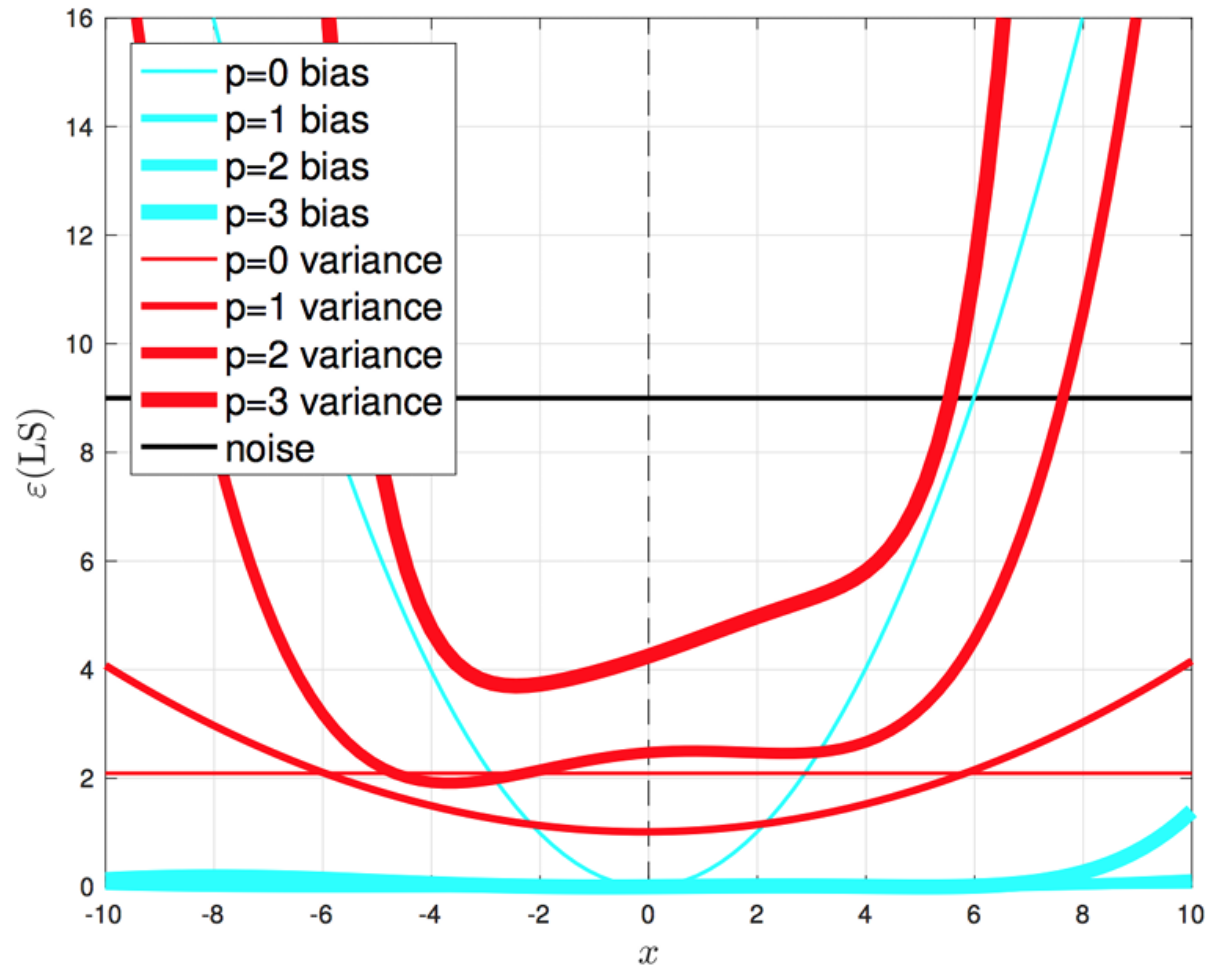


Bias and Variance: $\text{deg} = 2$

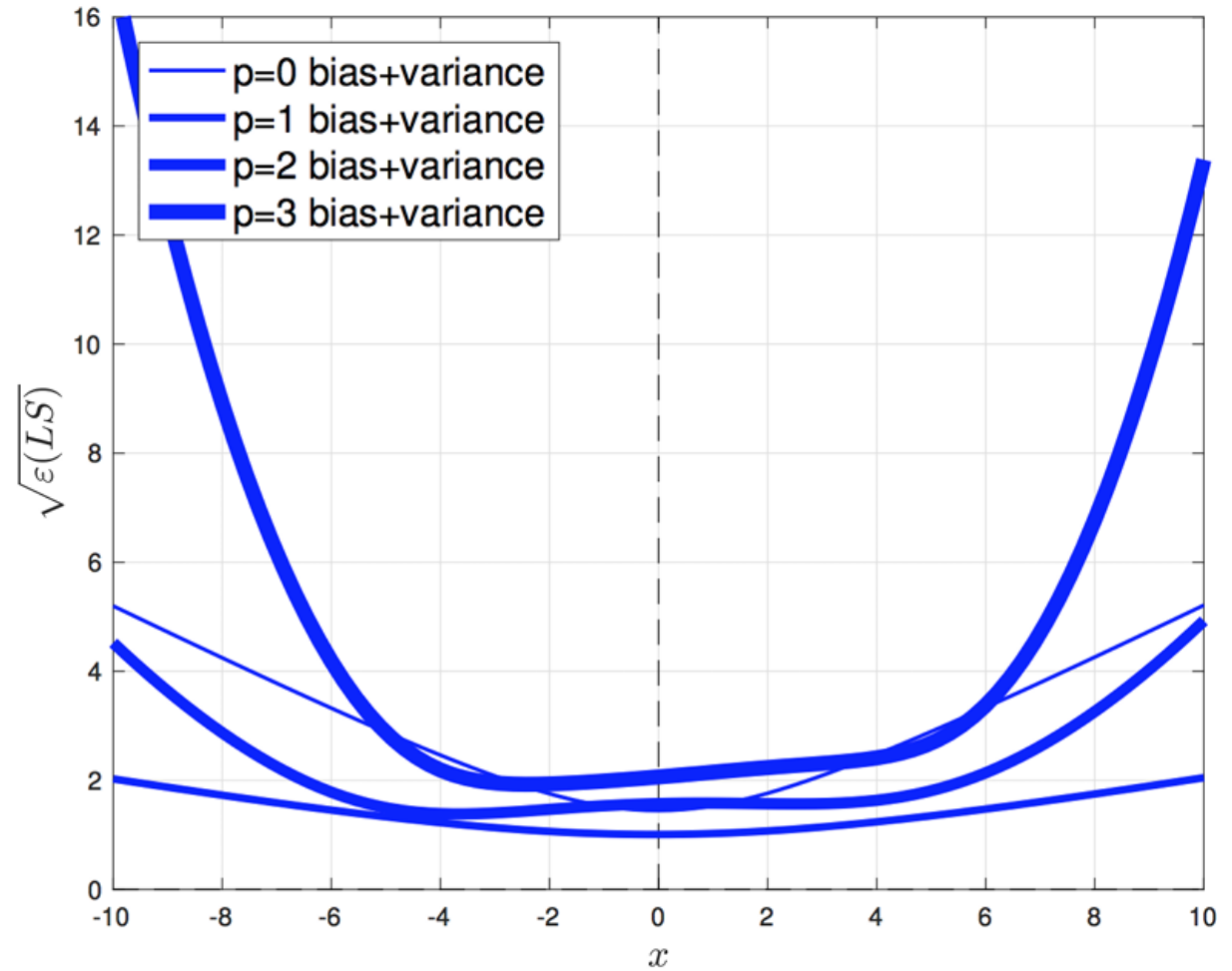


Experiments: Results

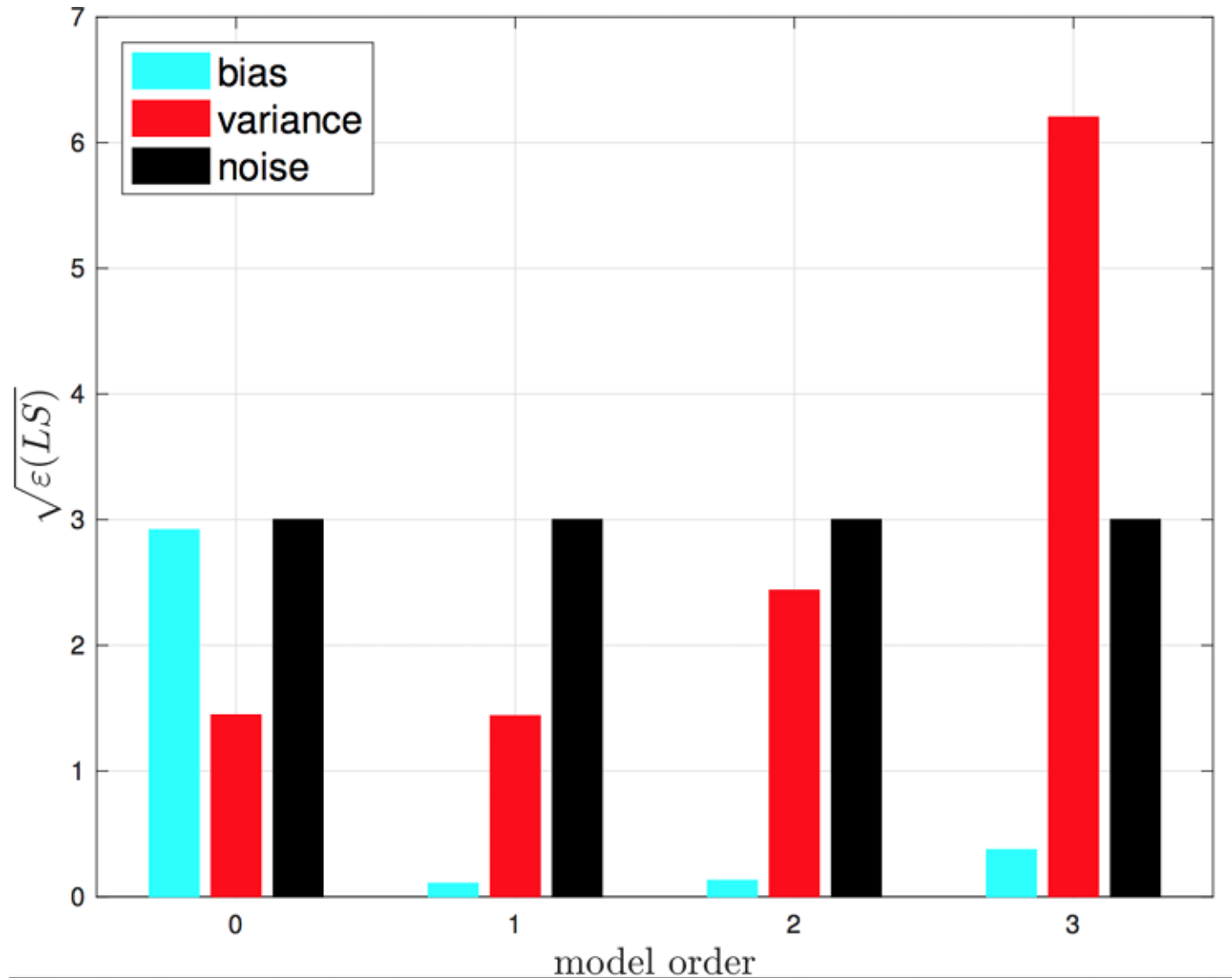
Variation of Bias/Variance Over Model Order



Variation of Prediction Error with Model Order



Bias and Variance: Underfitting vs Overfitting



Some Implications

1. Underfitting is equivalent to high bias; most overfitting correlates to high variance
2. Training error reflects bias but not variance. Test error reflects both. In practice, if the training error is much smaller than the test error, then there is overfitting
3. Variance $\rightarrow 0$ as $n \rightarrow \infty$
4. If f is in the set of hypothesis functions, bias will decrease with more data. If f is not in the set of hypothesis functions, then there is an underfitting problem and more data won't help
5. Adding good features will decrease the bias, but adding a bad feature rarely increase the bias
6. Adding a feature usually increase the variance, so a feature should only be added if it decreases bias more than it increases variance
7. Irreducible error can not be reduced
8. For real-world data, f is rarely known, and the noise model might be wrong, so we cant calculate bias and variance. But we can test algorithms over synthetic data